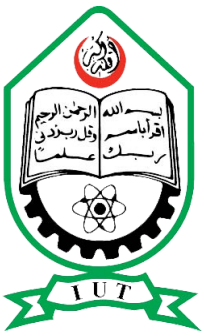
ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)



Final Report

SWE 4404: Software Project Lab II

Project Title

WanderVille

Name of the Supervisor

Md. Tariquzzaman

Team Members

Safia Aman Anika	230042120
Musfeka Ferdousi	230042126
Khadiza Sultana	230042135

Source code: github.com/SafiaAmanAnika/Wanderville

Table of Contents

Table of Contents **2**
1. Executive Summary **4**
2. Problem Domain and Background **4**
2.1 What kind of game is this? 4
2.2 The problem we chose to solve 4
2.3 Programming ideas used 5
3. Literature and Existing Solutions **5**
4. Requirements (SRS Session) **6**
4.1 People and needs 6
4.2 Main functional requirements 6
4.3 Main non-functional requirements 6
4.4 Requirements linked to user needs 7
4.5 SRS feedback and sustainable design 7
5.1 The whole system 7
5.2 Why we separated the server into folders 8
5.4 Database design 9
5.5 Why not another architecture? 9
5.6 Design session and later changes 10
6. Implementation Session **10**
6.1 Project folder structure 10
6.2 What runs in Godot? 10
6.3 What runs on the Python server? 11
6.4 One complete example: planting a seed 11
6.5 How crop growth works 12
6.6 Inventory, Aura and the store 12
6.7 Levels and rewards 12
6.8 Social lobby, friends and private farms 13
6.9 Networking and deployment 13
6.10 Why the classes are divided this way 13
6.11 Method and class sizes 14
6.12 Work revised after implementation 14
7. Tools, Version Control, Testing, and CI/CD **14**
7.1 Why we used these tools 14
7.2 Version control 15
7.3 What tests actually passed 15
7.4 Automation 15
8.1 Account and player data 15
8.3 Risks and unfinished policies 16
9. Project Management and Feasibility **16**
9.1 Project stages and changes 16
9.2 Was the project practical? 16
10. Individual Contributions **17**
11. Results, Limitations, and Future Work **18**
11.1 What is in the final main-branch code? 18

11.2 Known limits 18
11.3 Future improvements 18
12. Glossary and References **19**
12.1 Small glossary 19
Appendix A. Functional Requirements (FR 1–126) **20**

1. Executive Summary

Wanderville is a small online farming game built for players who want to relax in a 2D world. A player signs in, chooses a character and a pet, enters a colourful 2D world, grows crops, sells produce for Aura (the in-game money currency), and earns experience points to unlock levels and new items. The game also has a social lobby, private messaging system.

The problem is simple: if a game saves everything only on the player's computer, players can lose their progress when they switch devices, and two people may not see the same changes. Players could also change their money or item counts locally. We wanted the game to keep one reliable record that players could return to.

We built two connected parts. Godot draws the map, characters, menus and animations. A Python server built with FastAPI receives requests and checks the rules. PostgreSQL saves accounts, farm tiles, crops, items, Aura and social information. For example, the server checks that a player owns a seed, has unlocked it and is planting in a valid season before removing the seed and saving a crop.

The game uses normal web requests for actions such as buying and harvesting. It uses a WebSocket (an open two-way connection) for movement, online presence and nearby communication. The main branch includes code for separate social-lobby and private-world rooms. A separate Godot ENet host can check admission tickets, but it is not the default movement system.

CO6 — Can the project be explained to a non-technical reader?

Yes. The central idea is that Godot shows the game, the server checks actions and PostgreSQL remembers what happened. Later sections use examples before introducing technical names.

2. Problem Domain and Background

2.1 What kind of game is this?

Wanderville belongs to the farming and social-simulation game category. The main users are people who want short, relaxed play sessions and time with friends. Players need clear controls, saved progress, fair rewards and private spaces. The teaching team needs a program that can be explained, tested and maintained.

2.2 The problem we chose to solve

A farming game has many connected changes. Planting removes a seed and creates a crop. Harvesting removes a ready crop and adds produce and experience. Buying an item reduces money and adds inventory. If any one of these changes is saved without the others, players could lose items or receive rewards twice.

The social part introduces another concern. A stranger must not be able to enter a private farm just by knowing its number. The server therefore checks who owns each world and who is allowed to visit.

2.3 Programming ideas used

Idea	Plain meaning	Where we use it
Dictionary / list	A way to group values, such as a player and their items.	Godot keeps local menu data; Python builds responses.
Class / object	A small piece of code responsible for one part of the game.	FarmingService checks farming; InventoryService handles items.
Database transaction	A group of changes that must all succeed or all fail.	Planting saves the crop and spends one seed together.
State / rules	An action is allowed only in the right situation.	A crop must be ready before harvest; an invite must be accepted before a visit.
Client/server communication	The game sends a request; the server checks it and replies.	Login, planting, buying, messages and movement.

2.4 Why we need the database

PostgreSQL stores information that survives closing the game: users, worlds, crop locations, item stacks, currency, level, friendships and invitations. Relationships between tables help keep that information consistent. For example, each farm tile belongs to a saved world, and each world belongs to one player.

CO1 — Which basic ideas solve the problem?

We use classes to keep features separate, lists and dictionaries to hold data, checks to enforce rules, and transactions to prevent half-finished changes. These ideas are visible in the farm, inventory and social services.

CO1 — What specialised knowledge was needed?

Game scenes and animation were needed to build the playable world. Web APIs were needed to connect Godot and Python. Database transactions were needed to protect money and inventory. Multiplayer rooms and login checks were needed so the correct people see the correct farm.

3. Literature and Existing Solutions

We looked at existing games to understand the kinds of experiences players already enjoy.

Example	What it shows	What we took from the idea
Stardew Valley	Farming, fishing, seasons, relationships and co-op play are familiar game loops.	Make farming the main repeatable activity and give players long-term goals.
Animal Crossing	Players build a personal place and can invite friends to visit.	Give each player ownership of a space and check visits separately.
Palia	A social life-simulation game combines activities with shared spaces.	Provide a lobby and social tools without turning the game into a competitive match.

Wanderville is not intended to match the size or content of these commercial games. It is a smaller course project focused on a clear server-backed farming loop, individual world ownership, a simple economy and social interactions.

For technical decisions we used official Godot, FastAPI, PostgreSQL and SQLAlchemy documentation. The Godot networking guide advises developers not to trust a player's device with important shared data. That advice matches our decision to check inventory, ownership and money on the server.

CO2 — Does this work build on existing solutions?

Yes. Familiar farming and social-game ideas guided the design. We made the implementation smaller by using one Python server with separate feature modules, instead of running many separate services.

4. Requirements (SRS Session)

4.1 People and needs

Person / group	What they need
Player	A simple game that saves crops, money and progress; easy movement and social controls.
Player hosting a farm	Control over invitations and who can enter the world.
Development team	Readable files, tests and repeatable database changes.
Course evaluator	Evidence that requirements, design and code match.
Backend operator	A way to start the server, check health and recover from errors.

4.2 Main functional requirements

Functional requirements describe things the game must do. The source SRS contains 126 numbered functional requirements, represented by 147 user-story rows. The full identifier list is included in Appendix A so readers can check one requirement at a time.

Group	User action	Where it is implemented
Account	Sign up, log in, verify email, reset a password.	auth, profile and Godot login scenes
Farming	Till, plant, water, harvest and view crops.	farm_controller.gd and farming service
Inventory and shop	See item stacks, spend Aura, buy and sell.	inventory, economy and shop modules
Progress	Gain XP and unlock items by level.	progression service and HUD
World	Move, load the right world and see other players.	world, realtime and Godot scenes
Social	Search players, friends, direct messages and visits.	social module and lobby scripts
Other activities	Fishing, pets, housing, events and notifications.	Related backend modules; client coverage varies

4.3 Main non-functional requirements

Non-functional requirements describe how the game should behave: protect accounts, preserve data, recover after a connection problem, remain understandable and run on the resources available to the team.

Need	Concrete design choice	What still limits it
Data safety	One transaction for an item purchase or harvest.	PostgreSQL must be available.
Account safety	Hashed passwords and short-lived access tokens.	Real provider configuration must be tested.
Fair play	The server checks ownership, money and unlocked items.	Some movement physics checks are approximate.
Reliable purchases	A request ID allows a response to be checked after a timeout.	Requires receipt handling in the client.

Small hosting cost	One Python service and one database.	Free plans may sleep, limit usage or change.
Privacy	Friend blocks, private-world permission checks.	Formal retention and deletion policy needs team approval.

4.4 Requirements linked to user needs

Requirement IDs	Player need	Main evidence
FR 1–10; 122–126	Account, profile and recovery	auth routes; account scenes
FR 11–29	Movement and farming	world/realtime services; farming controller
FR 30–56	Activities, items and housing	fishing, inventory, shop, housing and pets
FR 57–79	Seasons, XP, tasks, events and alerts	world, progression, events and notifications
FR 80–102	Friends, chat, gifts and visits	social and realtime services
FR 103–121	Save, reconnect, settings and help	scene management, profile and state

4.5 SRS feedback and sustainable design

To reduce operating cost, we kept the server as one program with separate feature folders. This also avoids running several always-on servers for a small number of players. The project reuses 2D art assets and can run with a local PostgreSQL database while developing.

CO2 — Are functional and non-functional requirements distinct and linked to users?

Yes. Section 4.2 lists the actions, Section 4.3 lists quality and safety needs, and Section 4.4 links requirement IDs to the code areas. Appendix A gives the individual IDs. The original SRS remains the final source for its exact wording.

5. System Design (Design Session)

5.1 The whole system

The player runs the Godot game on a computer. Godot asks FastAPI to handle account and gameplay actions. The server checks the rules, reads and writes PostgreSQL, and sends a result back. WebSocket is used when information must keep arriving, such as movement and nearby players.

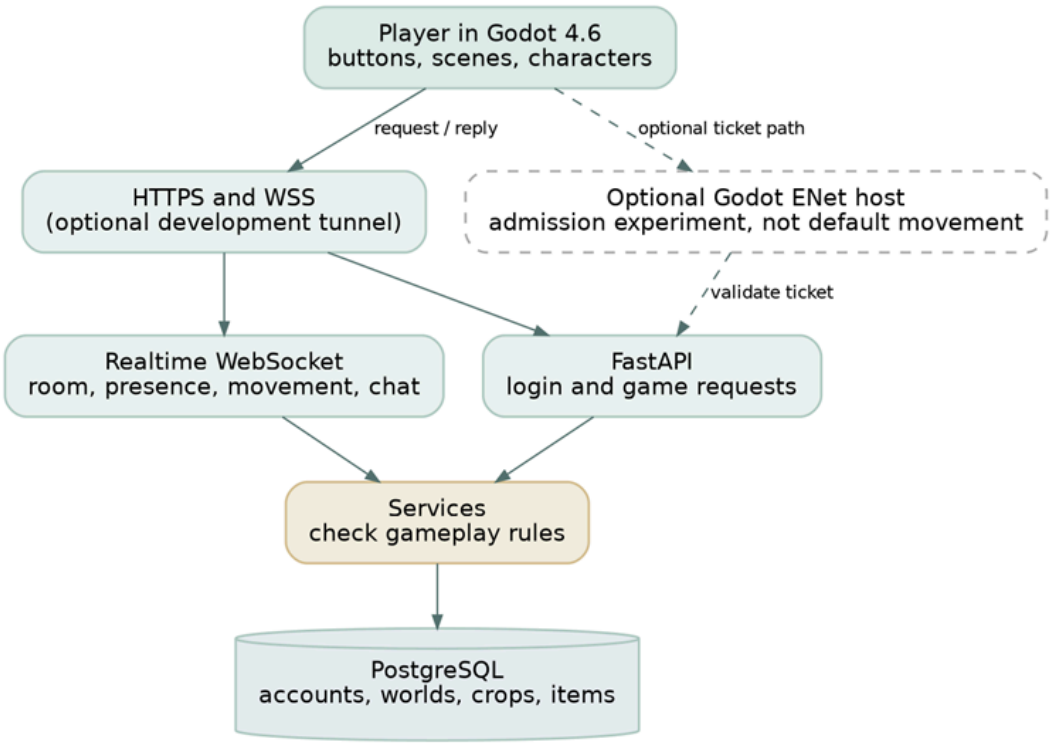


Figure 1. Wanderville system: one Godot client, one Python backend and one database.

5.2 Why we separated the server into folders

The server is one running application, but its code is divided by feature. Each module typically has a router (the URL), a service (the rule checks), a repository (database reads and writes), and models (table definitions). A single action travels down those layers. This makes a rule easier to find and a change less likely to break an unrelated feature.

Layer	Question it answers	Example
Router	Which request did the player make?	POST /farm/plant
Service	Is this request allowed?	Is this the player's tile and an owned seed?
Repository	Which database rows should be read or changed?	Find the tile and its inventory item.
Model	What is stored in a row?	PlayerWorld, FarmTile and InventoryItem

5.3 Separate worlds and the lobby

The server uses a different room identity for the social lobby, which is a place for players to meet. Each player also owns a saved world. A visitor cannot simply type another world ID and enter: the server checks whether an accepted invitation or ownership allows that room. A player's personal farm data is still stored under that world even when the owner is offline.

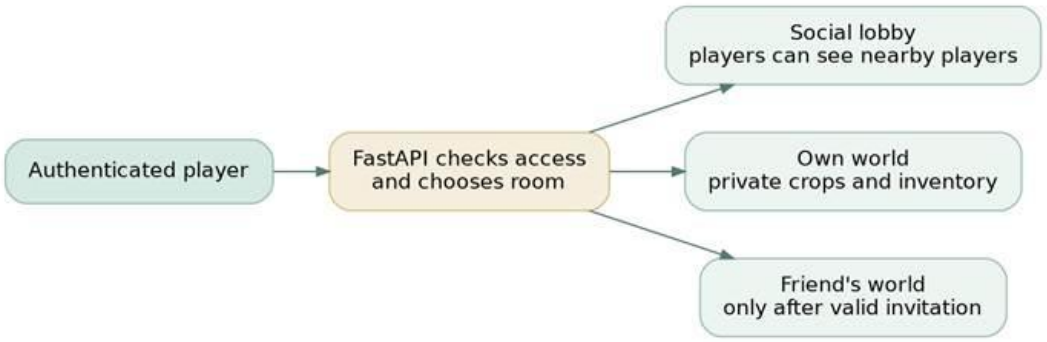


Figure 2. Room access is checked by the server rather than chosen freely by the client.

5.4 Database design

The database has many tables; Figure 3 shows only the most important relationships. Every player has an account. Each account owns one saved world, and that world has farm tiles. Inventory belongs to a player rather than to a picture on the screen. The same player record connects to money, experience and social records.

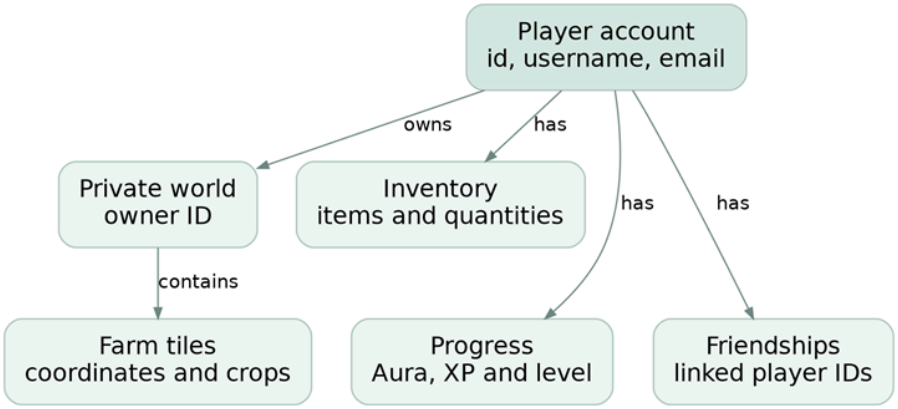


Figure 3. Simplified data relationships. This is not the complete 51-model schema.

5.5 Why not another architecture?

Option	Good part	Problem for our project	Choice
Everything stored in Godot	Fast offline play	Hard to protect items and support shared data.	Not selected
Several microservices	Independent teams can scale pieces separately	More servers, more deployment work and harder transactions.	Not selected
One FastAPI server with modules	One deployment, clear files and shared transactions	One worker is a scaling limit.	Selected

5.6 Design session and later changes

The design document originally described game objects such as Player, Farm, Inventory, House and Pet. In the code, some become database tables; others become Godot scene scripts or Python services. We also added receipt checks for shopping and separate visit sessions because a friend invitation and an actual visit are different things.

CO3 — Does the design match the requirements?

Yes in the main structure: account requirements go to auth, crops to farming, items to inventory and shop, social requests to social and realtime, and saved records to PostgreSQL. Figure 1 and the tables show these links. The final report does not claim that every feature has passed a complete live test.

CO3 — Were alternatives considered?

Yes. We compared client-only saves, several servers and one modular server. One FastAPI program gave the clearest balance of cost, safety and maintainability for this course project.

6. Implementation Session

6.1 Project folder structure

The source code is under the code folder, with the Godot game and Python server kept separate. The tree below leaves out large folders so the important parts are easier to see.

```
Wanderville/
   README.md                  how to set up and run the project
   render.yaml                proposed cloud start configuration
   .github/workflows/         native Opus extension checks
   code/
        client/
        project.godot         Godot project settings
        autoloads/            shared state and API connection
        Scenes/               menu, farm, lobby and market scenes
        Scripts/              farming, characters, UI and social code
        tests/                Godot checks
        server/
        app/
        main.py               FastAPI starts here
        core/                 settings, errors and shared rules
        db/                   sessions and database access
        modules/              auth, farming, inventory, shop, social, ...
        health/               health and readiness checks
        migrations/versions/   recorded database changes
        tests/                Python test suite
   docs/                      API, setup, database and test notes
```

6.2 What runs in Godot?

Godot is responsible for what players see and control. SceneManager.gd changes screens. GameState.gd stores the current view of the player's progress. APIClient.gd sends requests to FastAPI. RealtimeClient.gd keeps the WebSocket connection and updates nearby-player information. Individual scenes and scripts handle farming, the store, character animation, the social lobby and buildings.

6.3 What runs on the Python server?

FastAPI receives a request, reads the logged-in player and sends the action to the correct service. The farming service checks a crop action; the shop service checks the price and money; the social service checks friendships and invitations. Only the server is allowed to decide these saved values.

6.4 One complete example: planting a seed

Suppose a player has one wheat seed and clicks a prepared tile. Godot sends the tile coordinates and a seed item code. The server checks who sent the request, whether the player owns the world and seed, whether the tile is usable and whether the crop is allowed at that time. If every check passes, the seed count decreases and the crop is saved together. Godot then draws the growing crop using the result.

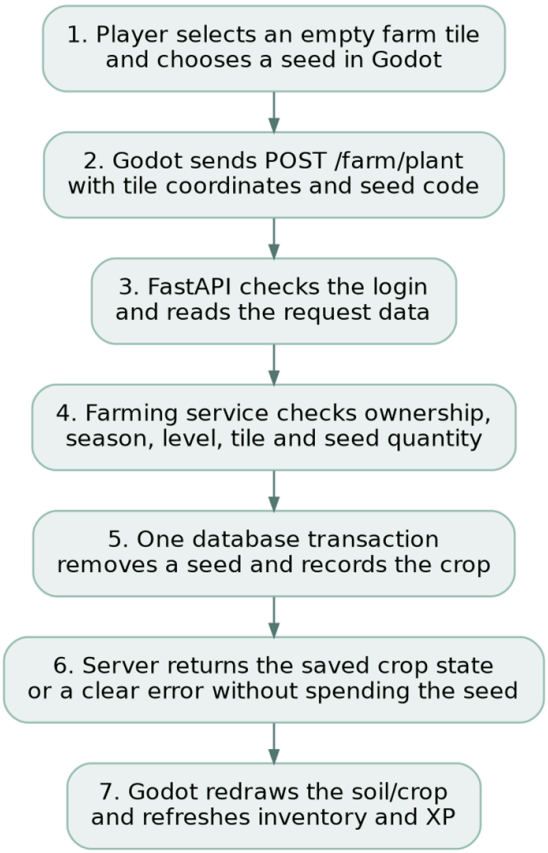


Figure 4. Planting is one server-checked action, from click to saved crop.

The planting route is POST /farm/plant. A small request contains x, y and item_code. A reply contains success or failure and data about the plot. The inventory and saved farm are then refreshed. The game should not remove the seed permanently before the server confirms that planting worked.

Message	Simple example	Meaning
Request	{"x": 3, "y": 4, "item_code": "wheat_seed"}	Plant at tile (3, 4) using an owned seed.
Success	{"success": true, "data": {...}}	The server accepted the action; use its saved result.
Failure	{"success": false, "error": "..."}	Show the reason; keep the previous saved items.

Exact accepted item codes depend on the active catalogue and compatibility routes; the live client builds the item code used by its current endpoint. The example illustrates the message format rather than substituting for an OpenAPI test.

6.5 How crop growth works

In the main branch, the farming service contains a path for the supplied crop sprites. A newly planted crop waits for watering. Its visual frames then advance through three timed intervals of 5, 6 and 7 seconds, reaching a harvestable frame after about 18 seconds. The backend also keeps a four-value growth-stage state (0 through 3); that number is not the same as the five drawings in a sprite sheet. Older catalogue items use a separate growth rule based on their stored duration.

The world model defaults to a 30-minute day-night cycle and a one-real-day season (24 hours). Weather and seasons are read from the server state. A separate old client-side timing constant exists in the repository; the current gameplay should take the server value as the source of truth.

6.6 Inventory, Aura and the store

Inventory items are saved as stacks. When the player collects more of an item, the server fills an existing stack first. The store sends a buy or sell request and receives the final result from the server. Shopping uses a request ID: if a connection fails after money is deducted, the client can check the receipt instead of repeating the purchase and paying twice.

New players can claim a one-time starter pack containing five Paddy seeds and five Wheat seeds. The claim is recorded so signing in again does not grant the pack repeatedly. Aura is the only game currency stored in the database; older names such as taka_balance are compatibility labels, not a second wallet.

6.7 Levels and rewards

Players earn experience (XP) through activities such as planting, harvesting, selling and fishing. Levels unlock certain items and increase storage. The documented progression has ten levels. For example, Paddy and Wheat seeds are available at level 1, seasonal crops at level 4 and special crops at level 8. The server checks these limits even if a player edits the local interface.

Level	Total XP needed	Title
1	0	Villager
2	80	Villager
3	180	Farmer
4	320	Farmer
5	500	Farmer
6	750	Settler
7	1050	Settler
8	1400	Settler
9	1800	Landkeeper
10	2100	Landkeeper

6.8 Social lobby, friends and private farms

After logging in, a player can enter the social lobby. The lobby has its own realtime room. RealtimeClient.gd asks the server to enter that room, and lobby_online_friends.gd reads room snapshots to show other player avatars.

A friend visit is different from an open lobby. The host invites a player, the server checks the invitation and visitor limit, and a visit session records who is inside. The player's own crop records do not become the host's crops. If a visit ends or permission is removed, the realtime client returns the player to an allowed world.

6.9 Networking and deployment

The normal network path uses HTTPS requests and a WebSocket. For local development, the team also used an ngrok address to let friends reach the backend running on a development computer. The repository contains a Render blueprint for a different, proposed cloud deployment with an external PostgreSQL database. Those are two separate environments and should not be described as one live production server.

There is an optional Godot dedicated host using ENet tickets. In this branch the default multiplayer transport remains the Python WebSocket route; the ENet path does not replace the normal movement system. Plain ENet is not suitable for public use without additional protection.

6.10 Why the classes are divided this way

Class / file	Main job	Why it is separate
APIClient.gd	Send web requests and handle replies.	UI scripts do not repeat the connection code.
RealtimeClient.gd	Keep the live connection, room and snapshots.	Movement changes frequently; it is not a normal HTTP request.
SceneManager.gd	Move between game screens.	A menu should not own all scene transitions.
FarmingService	Check and save farm actions.	Farming rules stay independent of the display.
InventoryService	Add, consume and stack items.	Buying, harvesting and gifts use the same item rules.
Social / FarmVisit services	Decide invitations, blocks and who can visit.	Access rules should not be duplicated in scenes.

6.11 Method and class sizes

Scope	Count	Largest (lines)	Average (lines)
Python functions / methods	1331	788	14.13
Python classes	212	503	33.00
Godot functions	1490	361	12.14

6.12 Work revised after implementation

The team separated farm invitations from active farm visits, added receipt recovery for the shop, combined common state reads into /state/bundle, and introduced a memory option for active-player positions. In the client, later work introduced a dedicated lobby room, improved

menu transitions and connected farming actions to saved server data. The revision record is in Git history and the project's docs folder.

CO1 — Where are the named programming concepts in code?

They appear in Godot dictionaries and scripts, Pydantic request validation, FarmingService rule checks, SQLAlchemy models, PostgreSQL transactions and realtime movement sequencing. The planting example shows how these ideas work together.

CO3 — Does the code follow the design?

Mostly yes: Godot remains responsible for presentation; FastAPI owns game rules and saved data. Some features grew beyond the early class diagram, especially visits, shopping receipts and realtime voice. Those additions stay in their own modules rather than replacing the basic architecture.

7. Tools, Version Control, Testing, and CI/CD

7.1 Why we used these tools

Tool	What it does	Why it fits here
Godot + GDScript	Draws the 2D world and controls scenes.	Good fit for existing pixel-art scenes and animations.
FastAPI + Python	Receives login, farm, shop and social requests.	Clear route and validation structure for a small team.
PostgreSQL	Keeps saved game data.	Transactions and relationships protect inventory and money.
SQLAlchemy + Alembic	Reads/writes rows; records schema changes.	Database changes can be repeated on a new installation.
Git + GitHub	Tracks change across branches.	A commit makes a change traceable.
pytest + Godot tests	Checks backend and selected client behaviours.	Catches errors before manual gameplay.

7.2 Version control

The team used Git as their version control for the project. The official repository for the project has 5 branches, including the main branch.

7.3 What tests actually passed

Test evidence	Limit
Backend tests on disposable PostgreSQL	Not a full rerun of the 29 Oct commit.
Additional migration and stack checks	SQLite cannot prove PostgreSQL lock behaviour.
Godot UI/gameplay checks	Used a lightweight stage, not the complete world.
Restart checks	Covered saved receipts and selected state.
Godot loading	Some rendering warnings still existed.

The test guide explicitly reports that some later full-world runs had native memory-allocation errors. A successful small UI test is therefore not the same as a successful two-player test on the entire map.

7.4 Automation

A GitHub Actions file builds and checks the native Opus extension on Windows and macOS. The Render configuration can install Python packages, apply migrations and start one Uvicorn worker. This does not mean the entire test suite runs on every commit; a full GitHub Actions workflow for Python tests and Godot checks is a future improvement.

CO4 — Why these frameworks, and what are the limits?

Godot and FastAPI let us keep the display and game rules separate. PostgreSQL makes saved changes safer than simple local files. The limits are one-worker realtime state, internet delays, a large world that is expensive to render, and provider requirements for login and email.

CO4 — Is version control and testing meaningful?

Yes. Code and documentation are in GitHub, and dated automated tests cover key farm, shop and inventory flows.

8. Professional, Ethical, and Legal Considerations

8.1 Account and player data

The server stores usernames, email addresses, password hashes, sessions and game records. It should never store a player's plain password. The code uses Argon2 for password hashing, access and refresh tokens for sessions, and server-side checks before exposing private data.

Google sign-in and sending verification emails are separate functions. Google identity is used for login; sending mail from the project's mailbox needs its own server-side permission. No private keys, tokens or email passwords belong in the GitHub repository or game download.

8.2 Keeping social features safe

Players can block one another, and the server checks invitation and room permissions. Private farm entry should always be decided by the server, not by a local scene variable. Text and voice features should be tested for permission changes such as blocking, leaving a room or losing a connection.

8.3 Risks and unfinished policies

Important limits remain: plain ENet connections are not encrypted for open internet use; in-memory presence and rate limits rely on one backend worker; and a full data-retention and account-deletion policy has not been approved. These should be addressed before inviting a large public audience.

The project contains custom and imported artwork, fonts and audio. The team must keep a record of these and replace the imported objects with custom ones in future.

CO4 — Were professional and legal concerns considered?

We can point to password hashing, token checks, private-world permission rules and bounded social features. We must also be clear that the asset licence register, retention policy and production-security review are incomplete. Those are requirements for broader deployment.

9. Project Management and Feasibility

9.1 Project stages and changes

Wanderville was developed across the SRS, design and implementation sessions.

Stage	Evidence	What the team should supply
SRS session	126 requirements and user-story mapping	Actual session date and feedback.
Design session	Earlier class and layered design documents	Actual session date and examiner feedback.
Core implementation	Godot scenes, FastAPI domain modules, migrations	Team meeting or task records.
Final integration	Git history, lobby/world and tunnel changes	Final two-client test record.

9.2 Was the project practical?

A small team can maintain this system because it uses one game engine, one Python program and one database. It can be developed locally..

Resource	Plan	Caution
Godot client	Runs on players' computers	Art and complete-map rendering need enough memory.
Python server	Local development; optional Render blueprint	One-worker state limits scaling.
PostgreSQL	Local or external managed database	Backups and migrations must be handled carefully.
Verification email	Configured Gmail API or Brevo option	Requires provider setup and quotas.

9.3 Changes in priority

The team focused on the actions needed for a playable demonstration: login, farming, inventory, shop, progression and movement. Larger ideas such as broad public multiplayer and full-world scaling were kept outside the final reliable scope. The system retained an optional ENet admission experiment rather than replacing an existing WebSocket path before it was fully ready.

CO7 — Is there evidence of planning and reviews?

Partly. Version-control history, migration files and dated test reports show repeated changes. The team must still attach its own task board, meeting notes or milestone log if it wants to claim formal task tracking.

CO7 — How were scope changes handled?

Since our project was game development, we started off with a fixed amount of features and scopes. Along the way, we added some other scopes that we thought fit our game well. We planned those scopes and then slowly implemented it step by step.

CO7 — Are costs and effort realistic?

The code has a viable low-cost local setup and optional free-tier services, but free hosting has quotas. Actual expenses and team hours need to be entered from team records.

CO7 — Can the work be completed with available resources?

The tested parts demonstrate a practical small-group prototype. Serving a much larger audience would need additional hosting, performance testing and security work.

10. Individual Contributions

Member	Student ID	Verified responsibility / evidence to insert
Safia Aman Anika	230042120	Map building, seasons change, day and night cycle, report writing and documentation..
Musfeka Ferdousi	230042126	Pet system, character system, farming, fishing, inventory, marketplace.
Khadiza Sultana	230042135	Entire backend system, server hosting, social lobby, multiplayer system, character customization system.

Coordination is visible in the shared codebase: client screens call backend routes, backend modules share a common database model, and changes to one feature require another part to use the new response format.

CO5 — Can each member show their own work?

Each member can show their part of the work that they did. However not everything can be shown on console. For example, Mushfeka did most of the game mechanics that can be directly seen in the screen, how each game mechanism works. Meanwhile for Khadiza, she did the full backend, and so, she can't directly show the backend since it's an inside mechanism, but she can tell how it works. Also, she did some of the game mechanisms that she can show on the game. Similarly for Anika who worked for the visual effects in the game as well as the map, her work is perfectly presented as the game loads.

CO5 — How were disagreements handled fairly?

For our team, disagreements rarely happened. We sat down together and discussed how each feature would go with diagrams and notes. Aside from that, our disagreements mostly happened due to stress and deadline approaching instead of actual reasons. We have been a good team so far and if we disagreed on something or weren't able to understand which way we should go, we consulted our supervisor who guided us.

CO5 — How did modules come together?

The Godot client and FastAPI communicate through defined HTTP requests and WebSocket messages. Shared response formats and server-side validation let separately written modules work together. Aside from that, each of us worked on a feature and launched them and so the modules were worked on separately and later merged together..

11. Results, Limitations, and Future Work

11.1 What is in the final main-branch code?

Feature group	Current evidence	Status for this report
Login and account recovery	Client scenes, auth routes and token logic	Code present; external-provider end-to-end test required
Farming and crop display	Farm controller and server farm routes	Code connected; dated gameplay checks
Inventory, Aura, shop and levels	Server modules, HUD, store and receipts	Connected; dated integration checks
Social lobby and realtime avatars	Dedicated room and lobby player code	Implemented in code; two-client check required
Invitation-based farm visiting	Visit-session and access-control services	Implemented in code; live isolation check required
Nearby text and voice	WebSocket proximity and voice modules	Code present; device/network checks required
Fishing, housing, pets and events	Backend modules plus selected client scenes	Coverage varies; not declared fully finished
ENet movement replacement	Admission bridge only in current configuration	Not the default movement transport

11.2 Known limits

- Full-world rendering and multiplayer performance can fail on computers with limited memory.
- The server knows useful map regions, but does not reproduce every Godot collision shape.
- Free hosting, tunnels and provider login require external configuration and stable network conditions.
- Running several backend workers needs shared storage for short-lived presence, voice and rate limits.

11.3 Future improvements

First, we need to run the full Python and Godot tests on their exact commits. Next, we need to test two players signing in from different networks, entering the social lobby, visiting a friend, returning to their own worlds, planting and harvesting without cross-world data leaks. Record the results and screenshots. After that, we need to fix remaining UI and animation issues, complete the missing client interactions, and prepare proper hosting, security and privacy policies if the project is made public.

CO6 — Can an informed reader understand the result and limits?

Yes. This report names the working code areas, the dated evidence and the activities needing more testing. It deliberately avoids claiming that every SRS requirement was completed end to end.

12. Glossary and References

12.1 Small glossary

Word	Meaning in this project
Client	The Godot game is installed on a player's computer.
Server	The FastAPI program that checks requests and saves game changes.
API	A named request such as POST /farm/plant.
WebSocket	A connection kept open for messages and player updates.
Database	PostgreSQL: the stored record of accounts and game progress.
Transaction	Several database changes saved together or cancelled together.
Room	A grouping of players who should see the same live place.
DTO / schema	The agreed format of data sent between client and server.
Migration	A recorded change to database tables.
XP	Experience points that increase level.
Aura	Wanderville's in-game currency.
ENet	A network transport used by Godot; optional admission path here.
CI/CD	Automatic code checks, builds and deployment steps.

12.2 Sources

[1] Wanderville repository https://github.com/SafiaAmanAnika/Wanderville

[2] Assessed commit
https://github.com/SafiaAmanAnika/Wanderville/commits/main/?before=538c8d540b50dce320f4c038ef060c9e7b08b2d2+35

[3] Wanderville README
https://github.com/SafiaAmanAnika/Wanderville/blob/main/README.md

[4] Wanderville backend structure
https://github.com/SafiaAmanAnika/Wanderville/blob/31ca97c39c059d0dc3d583da710da49731ad2448/docs/BACKEND_STRUCTURE.md

[5] Wanderville verification results
https://github.com/SafiaAmanAnika/Wanderville/blob/31ca97c39c059d0dc3d583da710da49731ad2448/docs/VERIFICATION_RESULTS.md

[6] Wanderville world models
https://github.com/SafiaAmanAnika/Wanderville/blob/31ca97c39c059d0dc3d583da710da49731ad2448/code/server/app/modules/world/models.py

[7] Wanderville realtime client
https://github.com/SafiaAmanAnika/Wanderville/blob/31ca97c39c059d0dc3d583da710da49731ad2448/code/client/autoloads/RealtimeClient.gd

[8] Godot Engine documentation https://docs.godotengine.org/en/stable/

[9] FastAPI documentation https://fastapi.tiangolo.com/

[10] SQLAlchemy documentation https://docs.sqlalchemy.org/

[11] PostgreSQL documentation https://www.postgresql.org/docs/

[12] Palia (official website) https://www.palia.com/en-US/

[13] Animal Crossing: New Horizons (official Nintendo site) https://animalcrossing.nintendo.com/new-horizons/share/

[14] Stardew Valley (official website) https://www.stardewvalley.net/

CO6 — Is the document suitable for its readers?

The report starts with the game in everyday language, introduces technical parts only when they are used, and includes figures, examples, references and this glossary so a reader can follow the implementation without knowing every framework beforehand.

Appendix A. Functional Requirements (FR 1–126)

The numbered titles below follow the functional-requirement list in the supplied original report. Status has a careful meaning: “Server code” is not the same as a complete game screen; “Client work” means the user-facing feature must be checked in Godot; “Partial” means its full behaviour was not confirmed.

ID	Requirement summary	Code evidence	Area
FR 1	Open game	Client work	auth / profile
FR 2	View title screen	Client work	auth / profile
FR 3	Create account; Handle duplicate username/email; Handle invalid password	Server code	auth / profile
FR 4	Log in	Server code	auth / profile
FR 5	Handle wrong password	Server code	auth / profile
FR 6	Stay logged in	Server code	auth / profile
FR 7	Log out	Server code	auth / profile
FR 8	Select character	Server code	profile
FR 9	Select map	Server code	profile
FR 10	Resume last selection	Server code	profile
FR 11	Enter world	Client work	realtime / world / client
FR 12	Move with keys	Partial	realtime / world / client
FR 13	Blocked by objects	Partial	realtime / world / client
FR 14	Camera follow	Client work	realtime / world / client
FR 15	Spawn at main gate	Partial	realtime / world / client
FR 16	Distinct world areas	Client work	realtime / world / client
FR 17	Landmarks on map	Client work	realtime / world / client
FR 18	Own farm world	Partial	farming / housing / progression
FR 19	Till farm tiles	Server code	farming / housing / progression
FR 20	Plant seeds	Server code	farming / housing / progression
FR 21	Water crops	Server code	farming / housing / progression
FR 22	See growth stages	Server code	farming / housing / progression

FR 23	No crop death	Server code	farming / housing / progression
FR 24	Harvest crops	Server code	farming / housing / progression
FR 25	Refill watering bucket	Server code	farming / housing / progression
FR 26	Empty watering bucket warning	Server code	farming / housing / progression
FR 27	View lifetime harvest count	Server code	farming / housing / progression
FR 28	Place outdoor decorations	Partial	farming / housing / progression
FR 29	Season-restricted planting	Server code	farming / housing / progression
FR 30	Fishing spots	Client work	fishing
FR 31	Catch fish	Server code	fishing
FR 32	Collect rare materials	Server code	fishing
FR 33	Catch to inventory	Partial	fishing
FR 34	Switch fishing rods	Server code	fishing
FR 35	Catch rare fishes	Server code	fishing
FR 36	Own house	Partial	housing
FR 37	Upgrade house	Server code	housing
FR 38	View partitioned inventory	Partial	inventory / shop / economy
FR 39	View XP and Aura	Server code	inventory / shop / economy
FR 40	Earn and hold Aura	Server code	inventory / shop / economy
FR 41	Sell for Aura	Server code	inventory / shop / economy
FR 42	Discard an item; Confirm before discarding item	Server code	inventory / shop / economy
FR 43	Inventory full warning	Server code	inventory / shop / economy
FR 44	View Aura history	Server code	inventory / shop / economy
FR 45	Buy from the shop; Insufficient Aura at shop	Server code	inventory / shop / economy
FR 46	Daily shop refresh	Server code	inventory / shop / economy
FR 47	Preview item before buying	Server code	inventory / shop / economy
FR 48	View collection log	Server code	inventory / shop / economy
FR 49	View fish log	Server code	inventory / shop / economy
FR 50	Adopt starter pet	Partial	pets
FR 51	Pet follows player	Client work	pets
FR 52	Feed the pet	Server code	pets
FR 53	Name the pet; Rename the pet	Server code	pets
FR 54	Pet rests when idle	Partial	pets
FR 55	Equip pet accessory	Partial	pets
FR 56	Play fetch with pet	Partial	pets
FR 57	Seasons change over time	Server code	world / worker
FR 58	Day and night cycle	Client work	world / worker
FR 59	Weather changes; Weather does not block play	Server code	world / worker
FR 60	Rain waters crops	Partial	world / worker
FR 61	Seasonal village décor	Client work	world / worker
FR 62	Track achievement progress	Server code	progression / notifications
FR 63	View in-progress and completed achievements	Server code	progression / notifications
FR 64	Claim achievement reward	Server code	progression / notifications
FR 65	Login streak resets	Server code	progression / notifications
FR 66	Level-up alert	Partial	progression / notifications
FR 67	Unlock content by level; See locked content	Server code	progression / notifications
FR 68	Track login streak	Server code	progression / notifications
FR 69	Give rewards on login	Server code	progression / notifications
FR 70	View daily tasks; Complete a daily task	Server code	progression / notifications

FR 71	Track task progress	Server code	progression / notifications
FR 72	Refresh daily tasks	Server code	progression / notifications
FR 73	Rare world event appears	Partial	events / worker
FR 74	Friend request alert	Server code	notifications / worker
FR 75	Gift alert	Server code	notifications / worker
FR 76	View notification list; Empty notification list	Server code	notifications / worker
FR 77	Dismiss notification	Server code	notifications / worker
FR 78	Live crop-ready alert	Partial	notifications / worker
FR 79	Unread notification badge	Server code	notifications / worker
FR 80	Search and add friends	Server code	social / realtime
FR 81	Accept or reject friends	Server code	social / realtime
FR 82	Visit friend farm	Partial	social / realtime
FR 83	Remove a friend; Confirm before removing	Server code	social / realtime
FR 84	Block self-request	Server code	social / realtime
FR 85	Prevent duplicate requests	Server code	social / realtime
FR 86	View friend profile	Server code	social / realtime
FR 87	Block a player; Unblock a player	Server code	social / realtime
FR 88	See online friend count	Partial	social / realtime
FR 89	Visit friend's map	Partial	social / realtime
FR 90	Set status message	Server code	social / realtime
FR 91	See friend's last-online time	Server code	social / realtime
FR 92	Send an invite; Invite expires	Partial	social / realtime
FR 93	Real-time movement	Partial	social / realtime
FR 94	Send and claim gifts	Server code	social / realtime
FR 95	Propose a trade; Cancel a trade proposal	Server code	social / realtime
FR 96	Accept a trade; Decline a trade	Server code	social / realtime
FR 97	Inventory updates after trade	Server code	social / realtime
FR 98	Proximity chat	Partial	social / realtime
FR 99	Send direct message	Server code	social / realtime
FR 100	Chat language filter	Server code	social / realtime
FR 101	Report a player	Server code	social / realtime
FR 102	Use an emote	Partial	social / realtime
FR 103	Directional walk animation	Client work	realtime / profile / client
FR 104	Idle animation	Client work	realtime / profile / client
FR 105	Action animations	Client work	realtime / profile / client
FR 106	Connection lost message	Client work	realtime / profile / client
FR 107	Resume after reconnect; Grace period expires	Partial	realtime / profile / client
FR 108	Auto-save progress	Partial	realtime / profile / client
FR 109	Manual save	Partial	realtime / profile / client
FR 110	Saves automatically	Partial	realtime / profile / client
FR 111	First-time tutorial; Tutorial steps in order	Partial	profile / client
FR 112	UI tooltips	Client work	profile / client
FR 113	Skip tutorial	Partial	profile / client
FR 114	First-encounter hint	Partial	profile / client
FR 115	Action sound effects	Client work	profile / client
FR 116	Audio settings	Partial	profile / client
FR 117	Toggle display mode	Partial	profile / client

FR 118	View game version	Client work	profile / client
FR 119	View controls help	Client work	profile / client
FR 120	Submit feedback	Server code	profile / client
FR 121	View credits	Client work	profile / client
FR 122	Request password reset; Invalid recovery email	Server code	auth / profile
FR 123	Change password; Wrong current password	Server code	auth / profile
FR 124	Delete account	Server code	auth / profile
FR 125	Confirm account deletion	Server code	auth / profile
FR 126	View account details	Server code	auth / profile